
SmartCliff
CAREER MOBILITY SOLUTIONS
We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

SmartCliff
Career Mobility Solutions

Getting Start with Python

APR, 2026

Getting Start with Python - Outline

- Introduction to Python
- Python Tokens
- Expressions
- Input and Output
- Type Conversion

INTRODUCTION TO PYTHON

What is Python?

✓ Python is an **interpreted, high-level, general-purpose, dynamic-typed,** and **object-oriented programming language.**

- **Interpreted:** Python code is executed **line by line by the Python interpreter**, rather than being compiled into machine code beforehand. This makes it easier to write and test code, as you can see the results immediately.
- **High-level:** Python is designed to be **easy to read and write**, with a **syntax** that emphasizes **readability.** This makes it accessible for beginners and reduces the cost of program maintenance.

What is Python?

- **General purpose:** Python is **not specialized** for a **particular type of application**. It can be used for a **wide range of tasks**, including **web development, data analysis, artificial intelligence, scientific computing, and more.**
- **Dynamically-typed:** In Python, the data type of a variable is **interpreted at runtime**. In **dynamically typed** languages like Python, you **don't need to specify the data type** of a variable when you declare it. Instead, the **type is inferred during the execution of the program**.
- **Object-oriented:** Python supports **object-oriented programming**, a paradigm that structures **code around objects**, which encapsulate data and behavior. This helps in organizing and structuring code in a modular way.

What is Python?

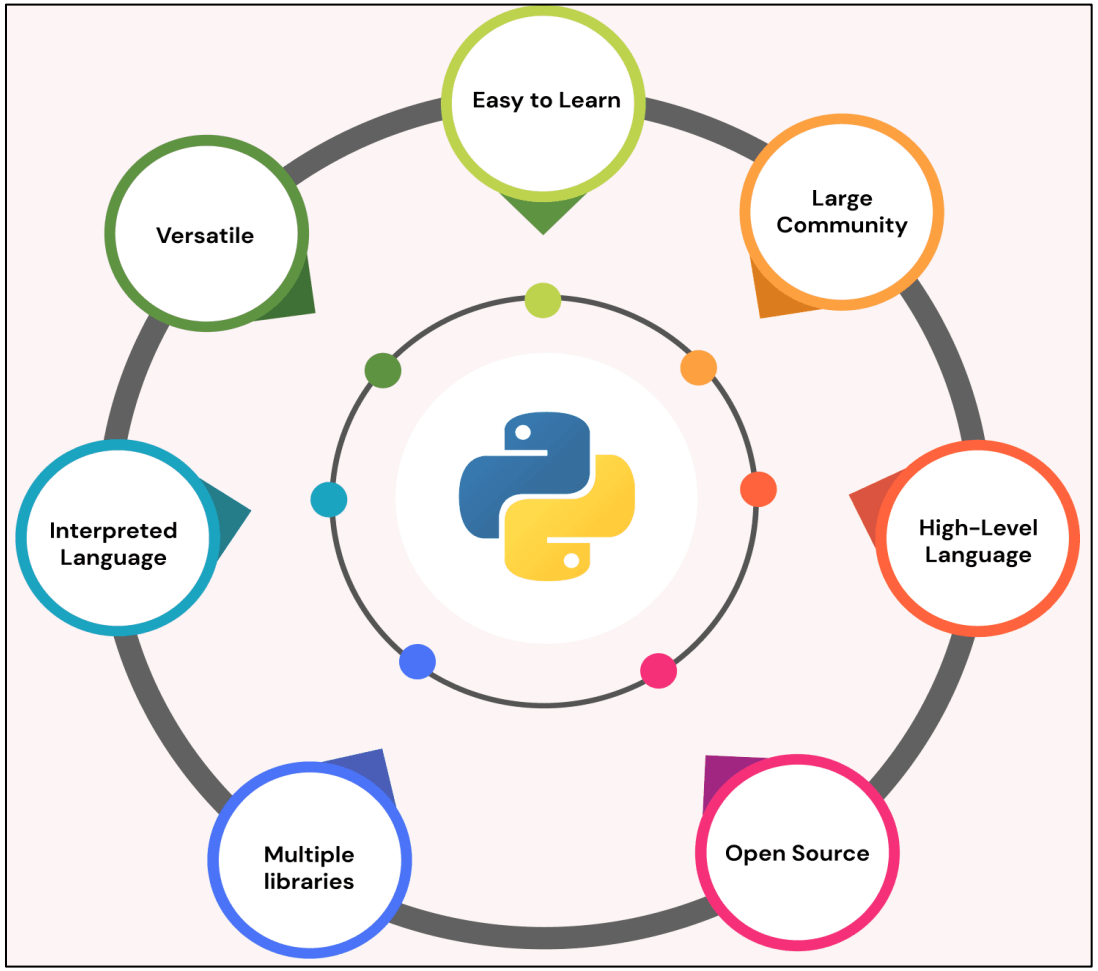
- ✓ **Python** was created by **Guido van Rossum**, and **first released in 1991** and developed under an **OSI-approved open-source license** (GPL, BSD), making it **freely usable and distributable**, even for commercial use.
- ✓ **Python's license** is administered by the **Python Software Foundation**.

Notes

Guido van Rossum named the language after the **BBC** show **"Monty Python's Flying Circus".** He doesn't particularly like snakes that kill animals for food by winding their long bodies around them and crushing them.

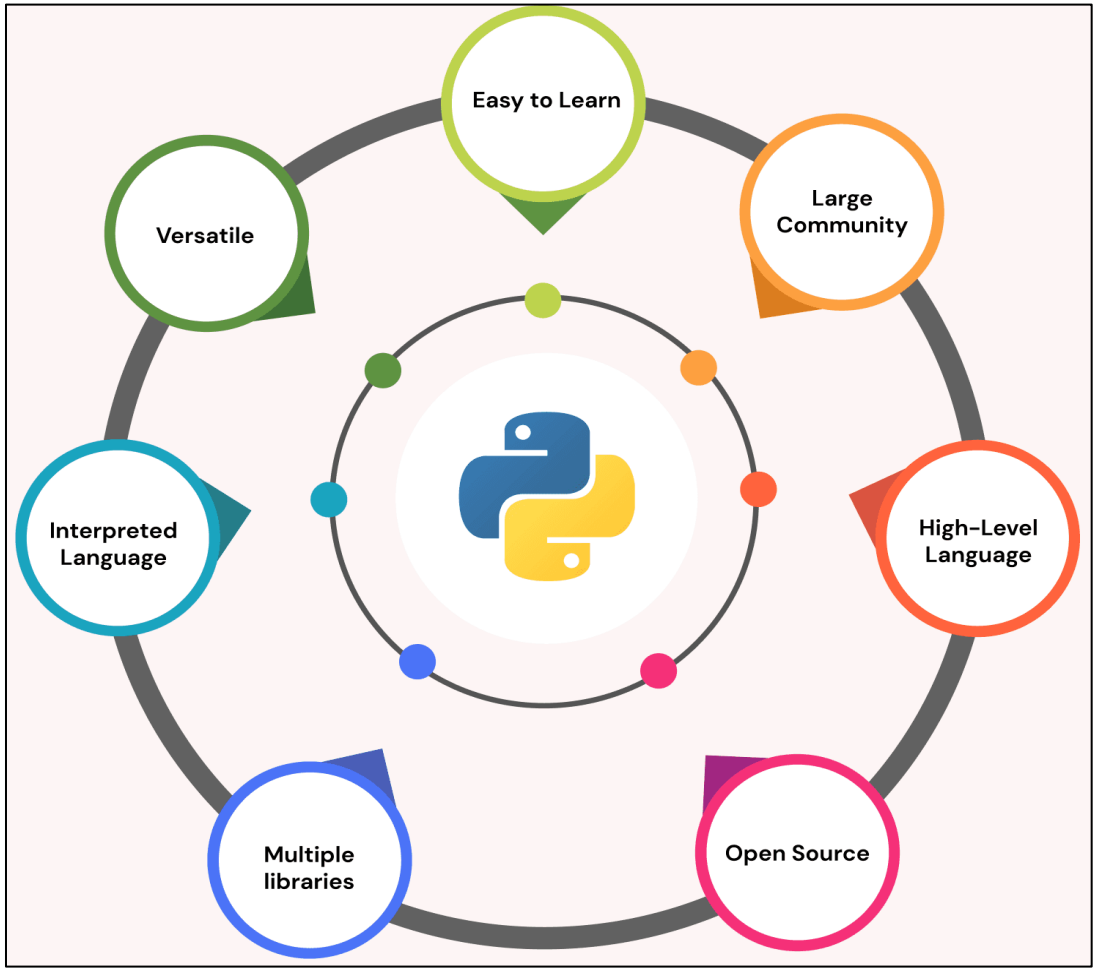
Why to Learn Python?

- ✓ **Easy to learn:** 👌 📖 Python is meant for its **simplicity and readability**, making it easy for beginners who are very interested in learning Python. The language has a **simple syntax similar to writing English**, and its code is often referred to as **"executable pseudocode,"** which is easy to read and understand.



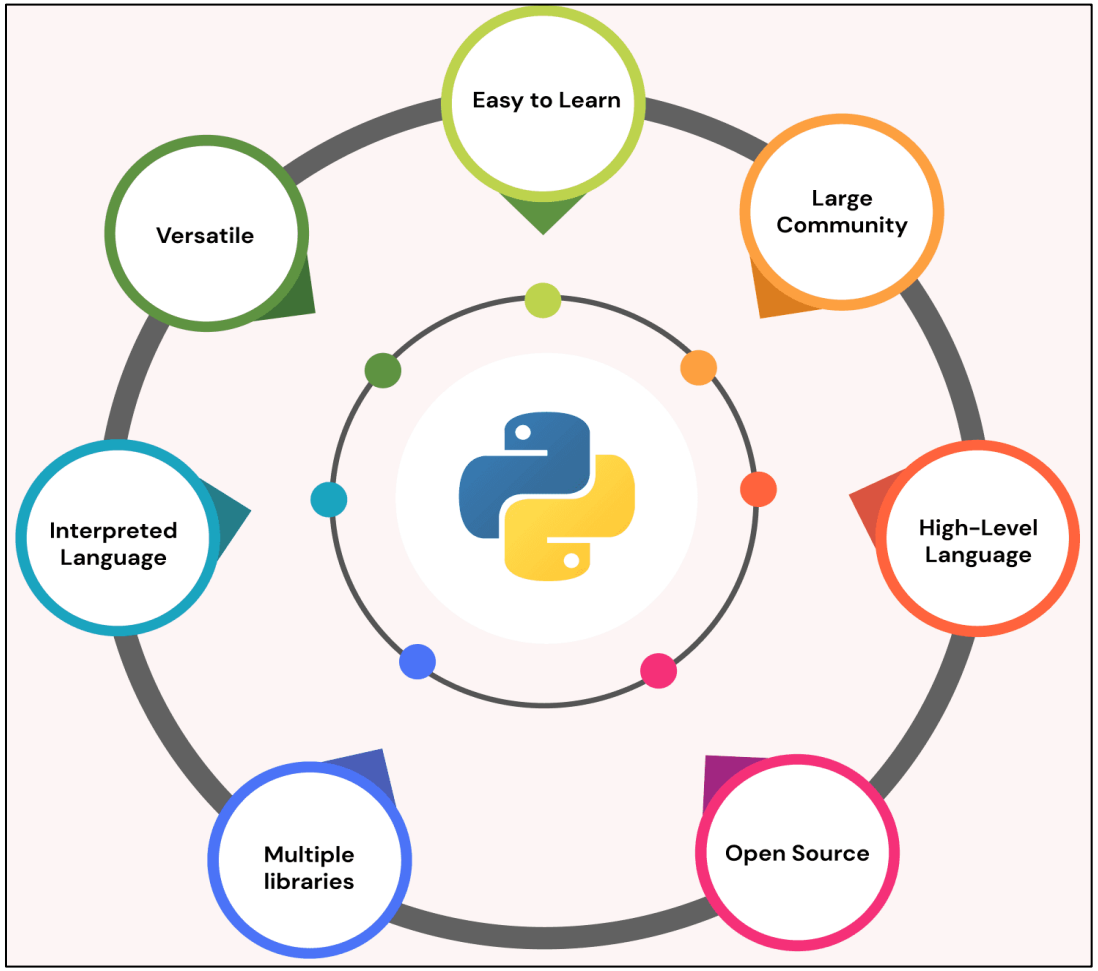
Why to Learn Python?

- ✓ **Versatile:** 🔀 Python is a programming language that can be used for **many applications** such as web development, data science, and machine learning. A **library** that is **standard,** along with **built-in functions** and **modules,** makes it possible to **write complex programs with minimal code**.



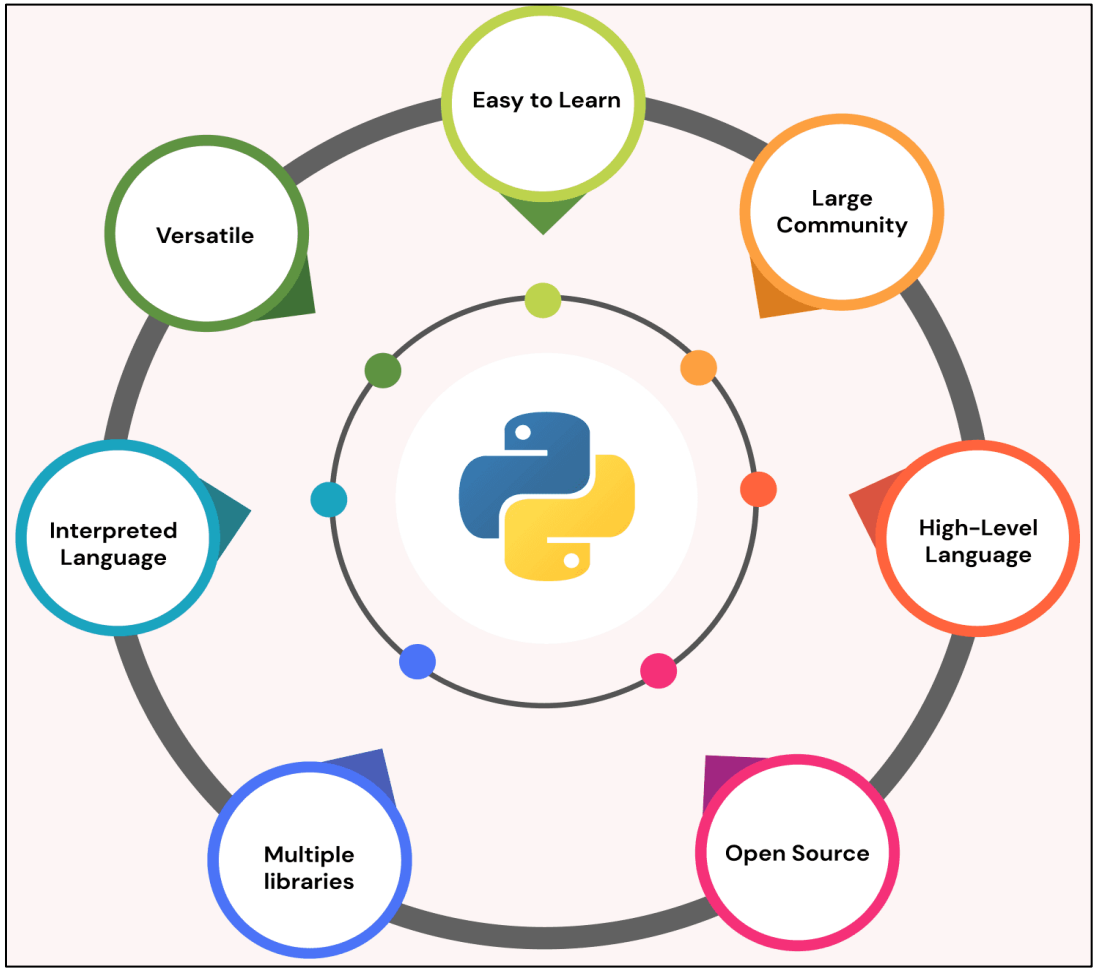
Why to Learn Python?

- ✓ **Large community**: 🌐 Python has a **large and active developer community,** with many **resources available** for learning and using the language. There are **many libraries** and **frameworks** for understanding advanced concepts.



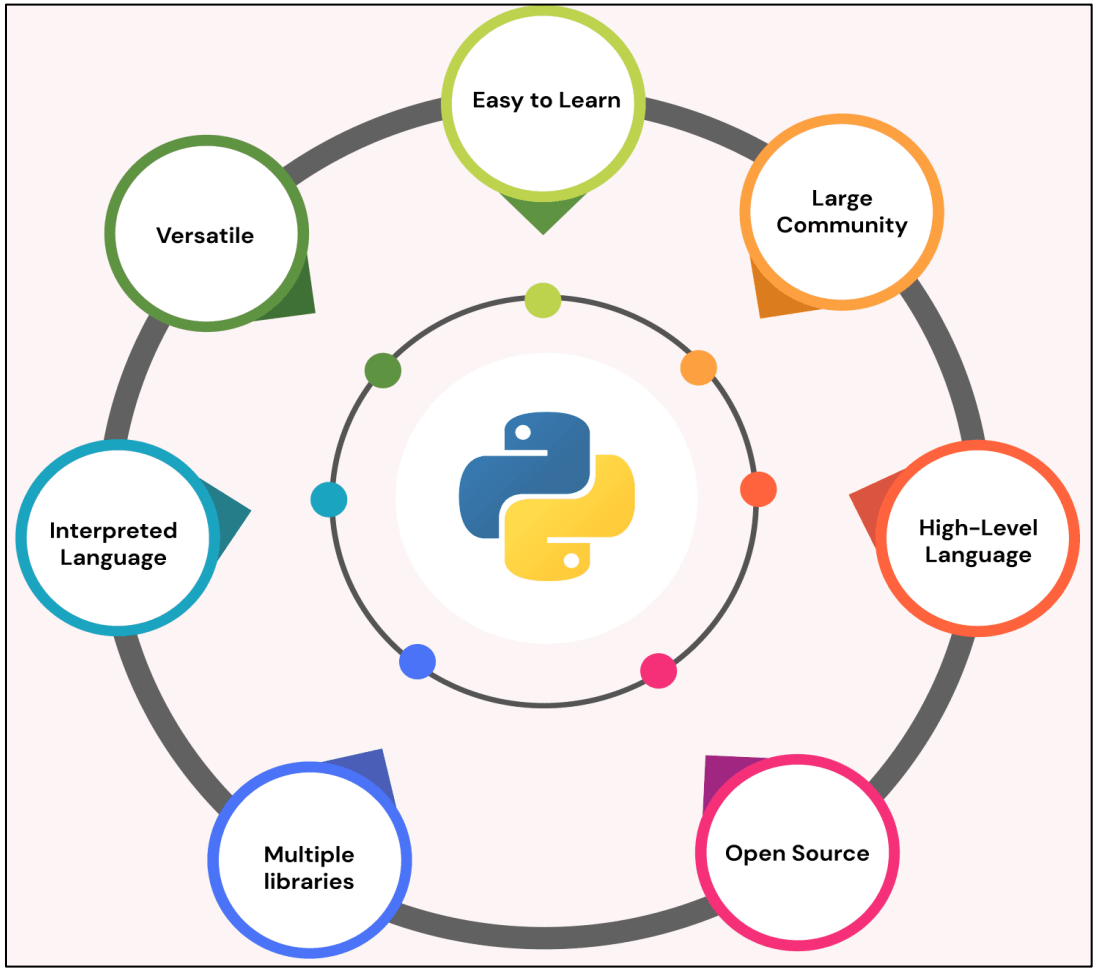
Why to Learn Python?

- ✓ **High-Level Language:** Python is considered a **high-level language**, which simplifies many low-level details of computer programming, like memory management and pointer arithmetic. Python makes **writing and reading code easier** and the **syntax of the code is easier to interpret by humans**



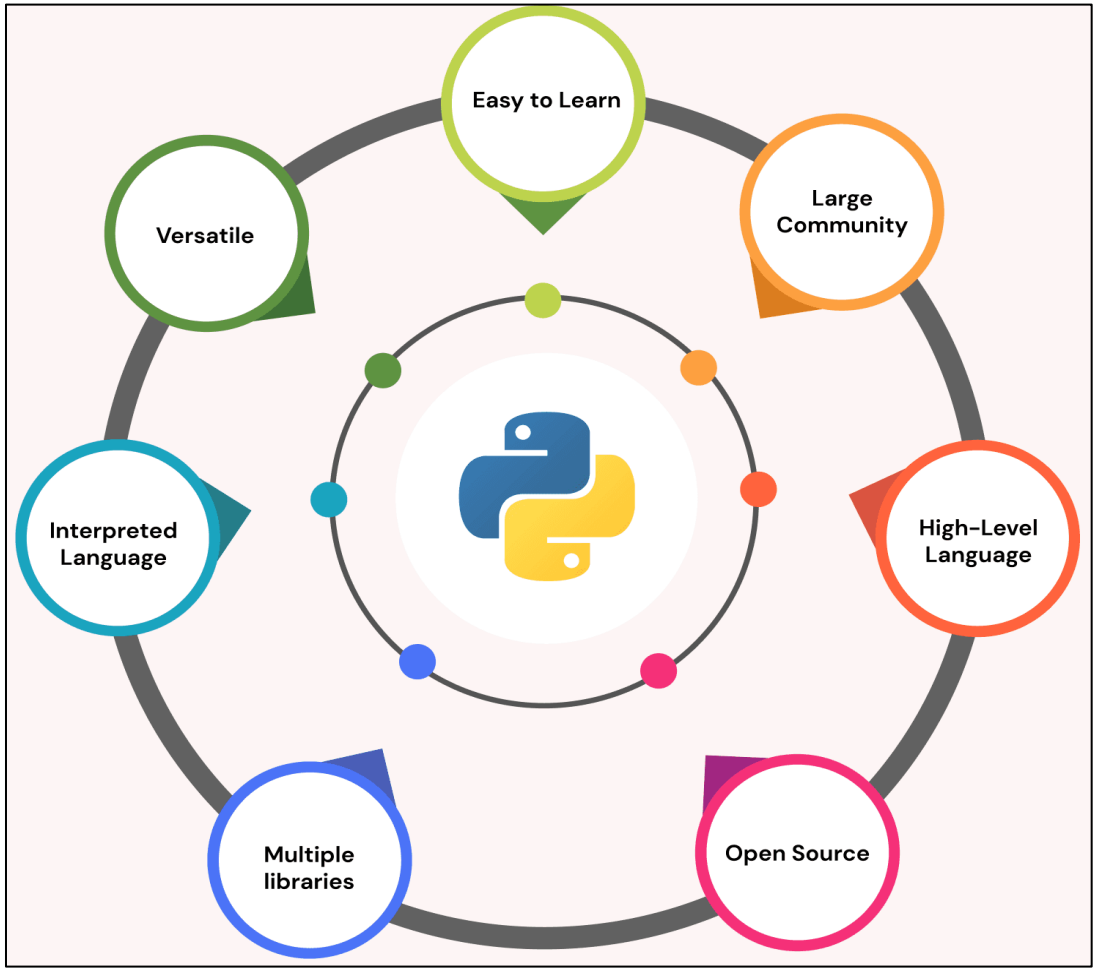
Why to Learn Python?

- ✓ **Open Source:** 🆓 Python is a language anyone can **use, change, and give away for free.** Many developers like to use it because they **can make it work just how they want it to.**



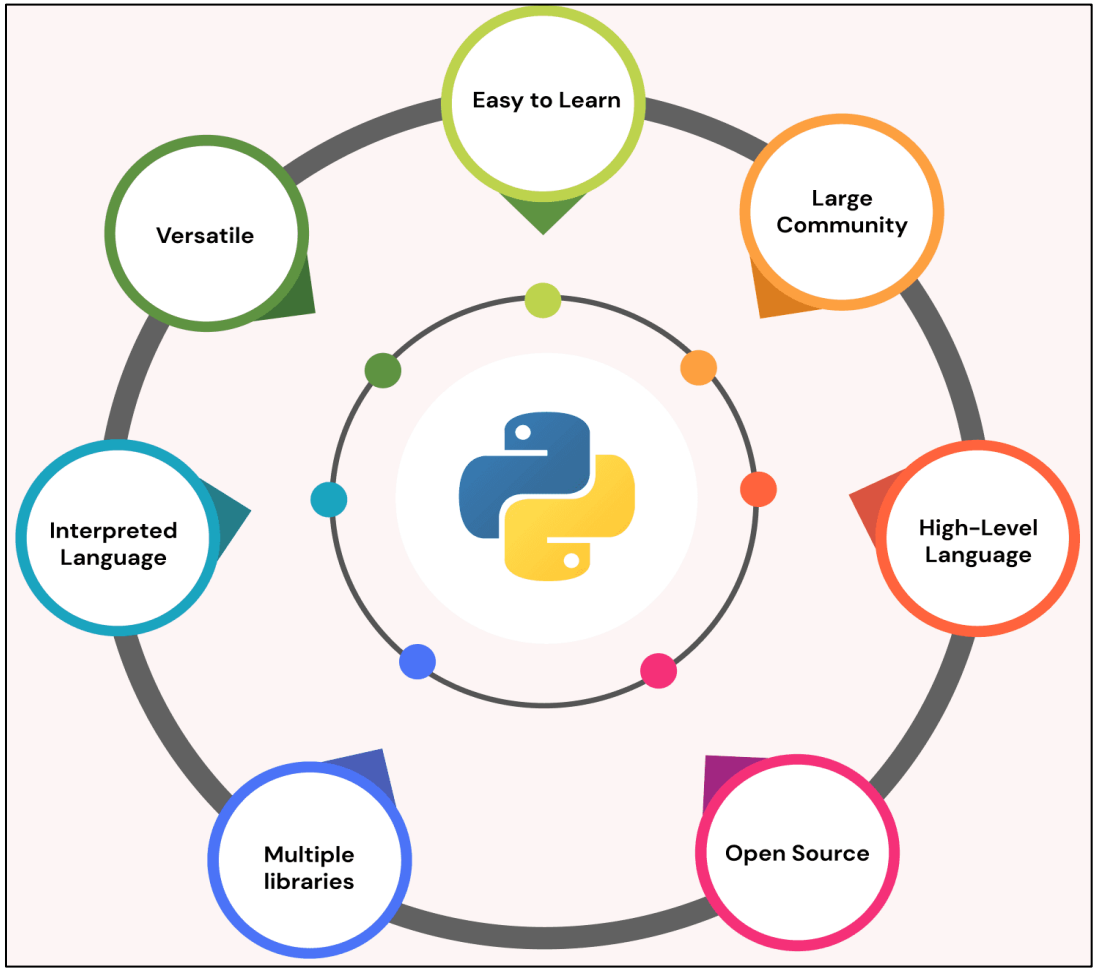
Why to Learn Python?

- ✓ **Interpreted Language:** 💻 Python is an **interpreted language** that can **run** directly from the **source code without compiling**. This makes **writing, testing, and debugging code easy** and can **speed** up the development process.



Why to Learn Python?

- ✓ **Multiple Libraries:** 📚 Python includes **numerous amount of libraries** that can be used just by **importing.**



Environmental Setup

- PyCharm is one of the most popular **Python-IDE** developed by **JetBrains** and used for performing **scripting in Python language**. PyCharm provides some very useful features like **Code completion inspection** and **debugging processes**.
- PyCharm provides various tools for productive development in Python.

How to run a simple Python Program?

Prerequisite: Before, starting with the process of Installing Python IDE PyCharm in Windows, one must ensure that Python is installed on their system. To check if the system is equipped with Python, go to the Command line(search for cmd in the Run dialog(window button + R).

- Now run the following command on cmd:

```
python --version
```

- If Python is already installed, it will generate a message with the Python version available.
- Otherwise, Install the Python from official page (with below link)

```
https://www.python.org/downloads/
```

How to run a simple Python Program?

Step 1: Go to website official website of Jetbrains and click on the “DOWNLOAD” link of the Community section.

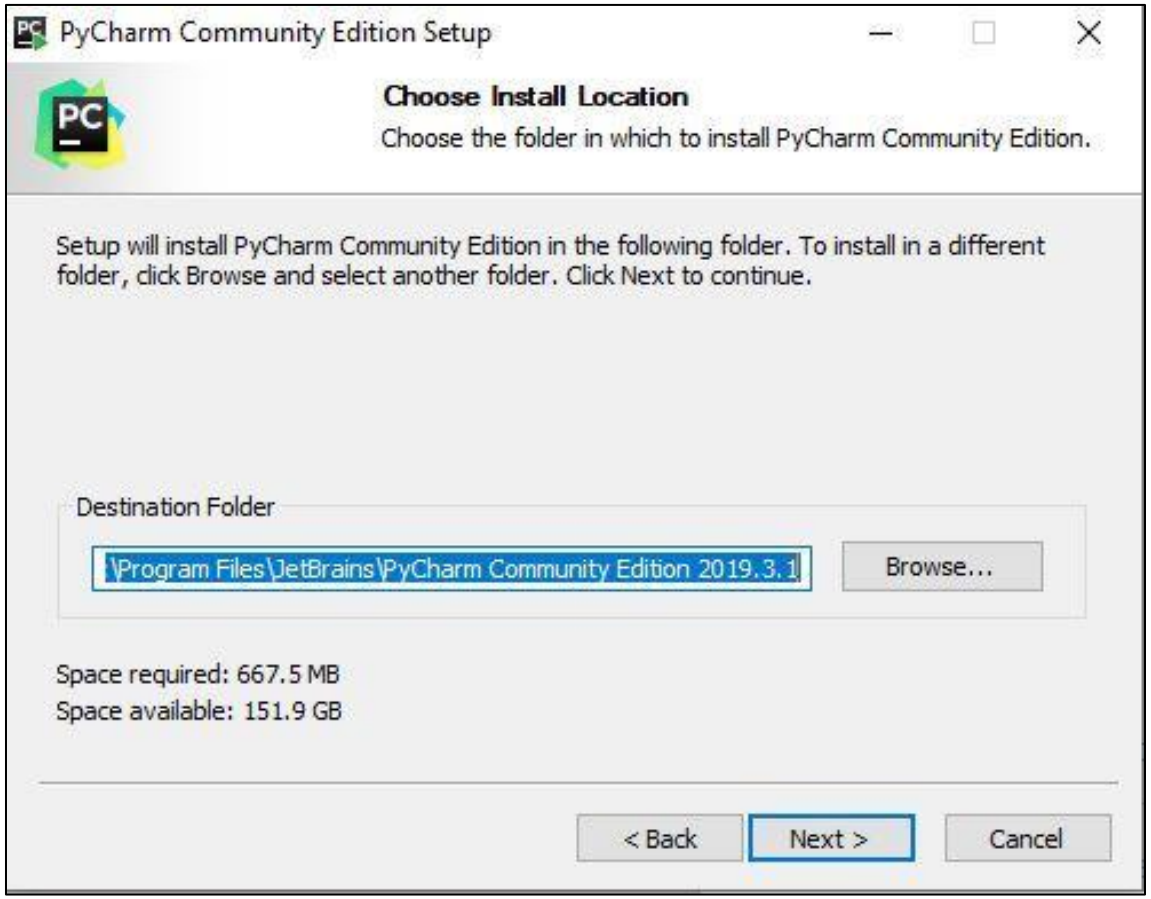
https://www.jetbrains.com/pycharm/download/

How to run a simple Python Program?

Step 2: After clicking on Download Click on Next.

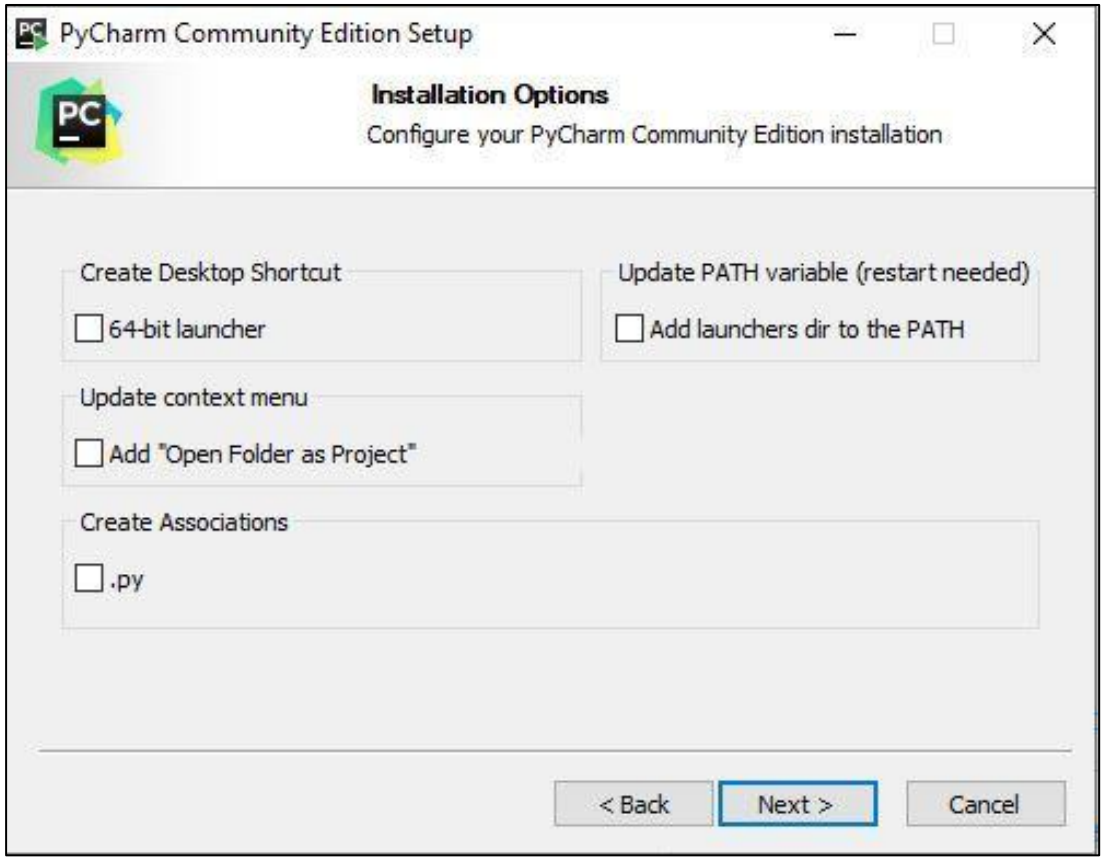
How to run a simple Python Program?

Step 3: After Click on Next , You need to choose the destination folder according to your choice.



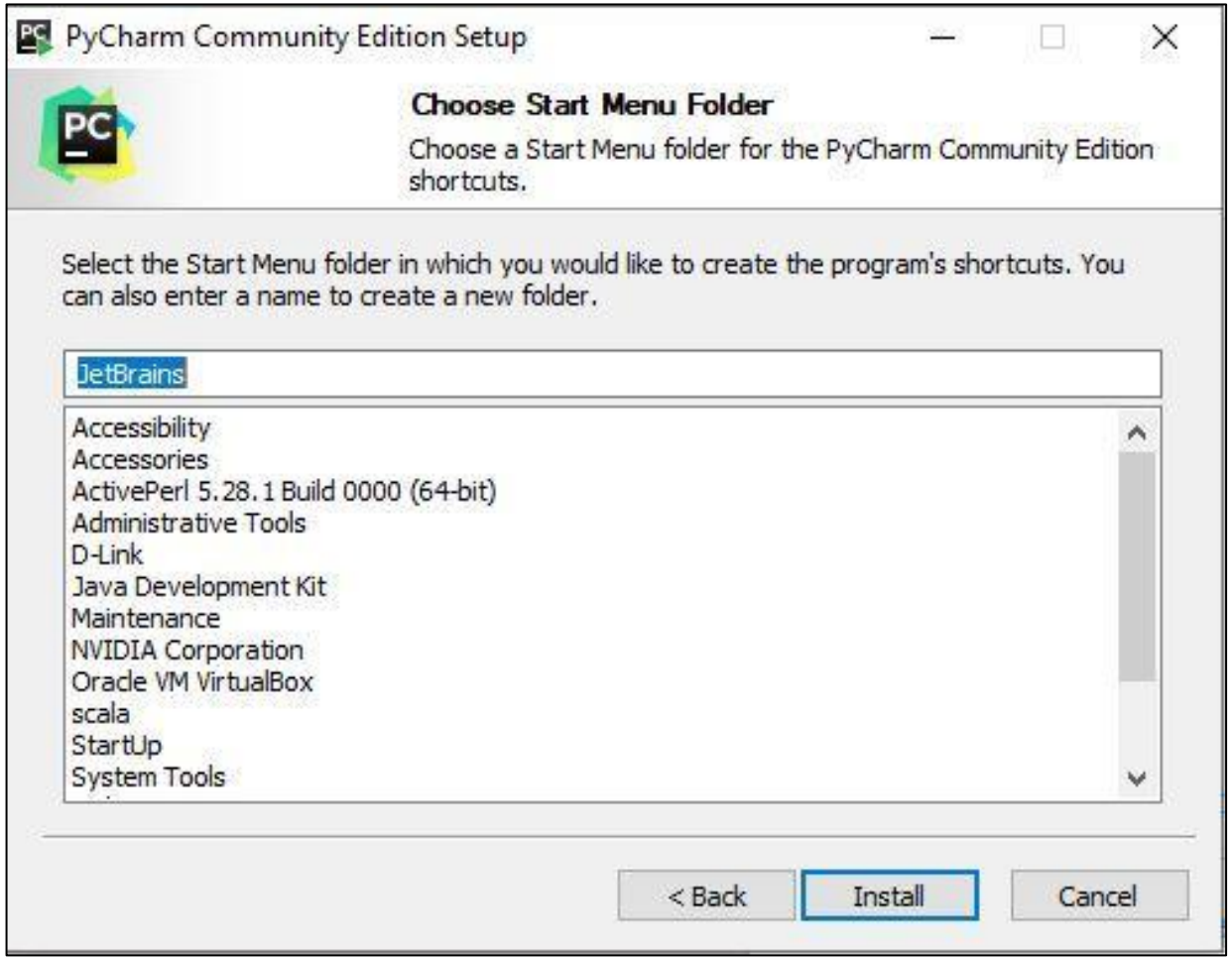
How to run a simple Python Program?

Step 5: Choose options of installation according to your choice.



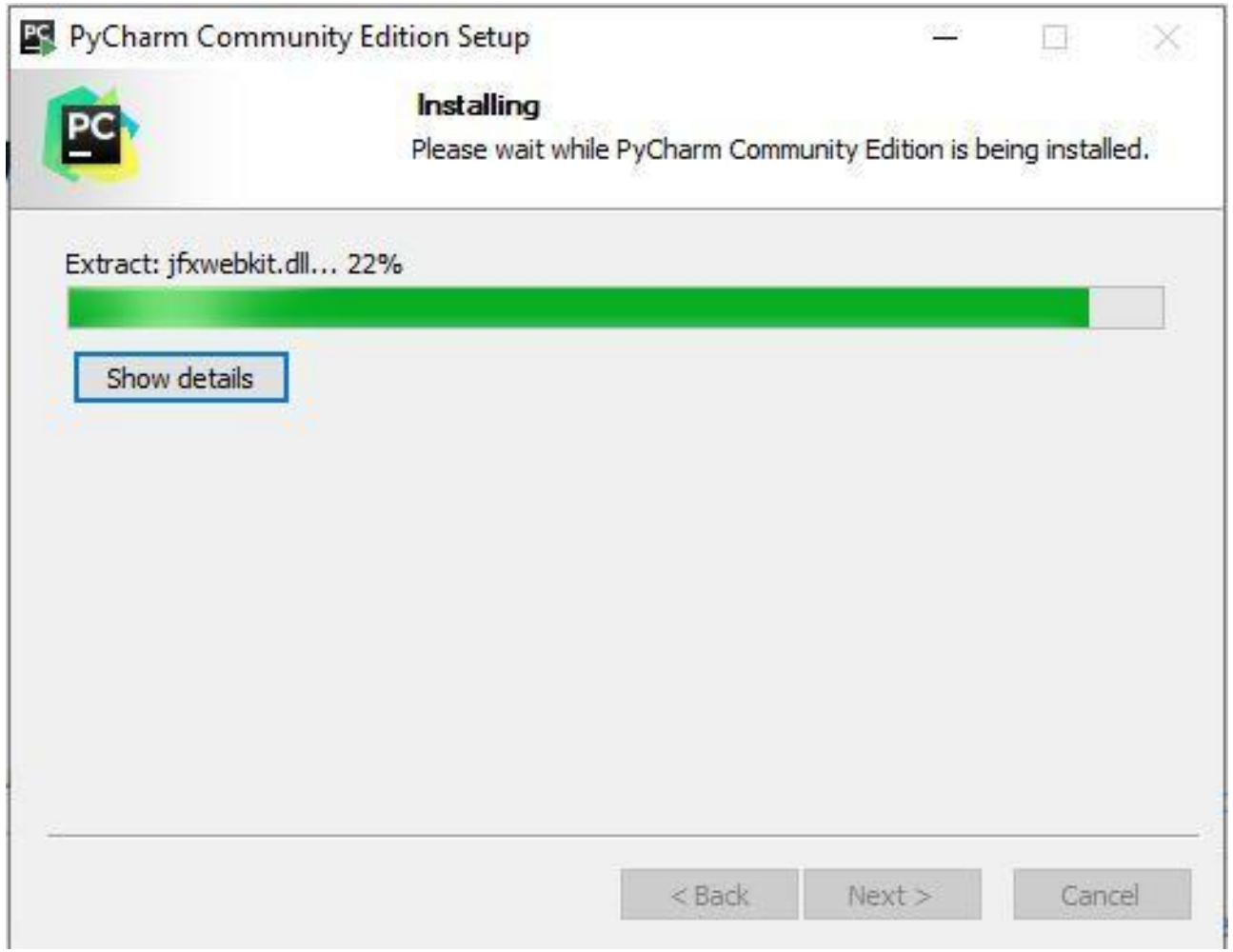
How to run a simple Python Program?

Step 6: Choose JetBrains and Click on “Install”.



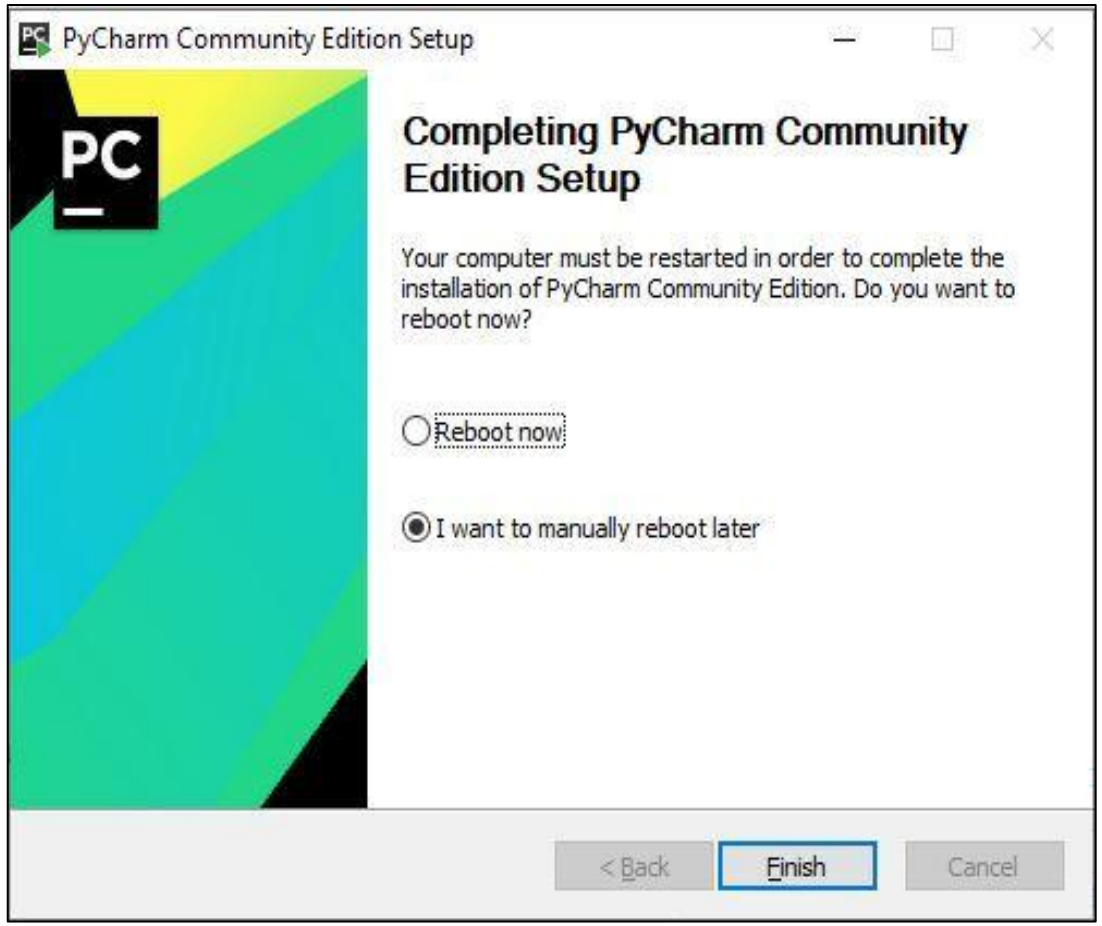
How to run a simple Python Program?

Step 7: Let the installation finished.



How to run a simple Python Program?

Step 8: After Installation completed , It will show that Pycharm is installed successfully , then click on “I want to manually reboot later”. Click on Finish and then the process is completed.



Comments

- ✓ In Python, **comments** are used to add **explanatory notes** or **annotations to the code**.
- ✓ Comments are **ignored** by the **Python interpreter** and are **intended** for **human readers** to **understand the code better**.
- ✓ There are two types of comments in Python as follows:

1. Single-line Comments:

- **Single-line comments** are created using the **# symbol** and can be placed on their **own line** or at the **end of a line** of code.

Example : Single-Line Comment

```
# This is a single-line comment
x = 5  # This is a comment at the end of a line
```

Comments

2. Multi-line(Block) Comments:

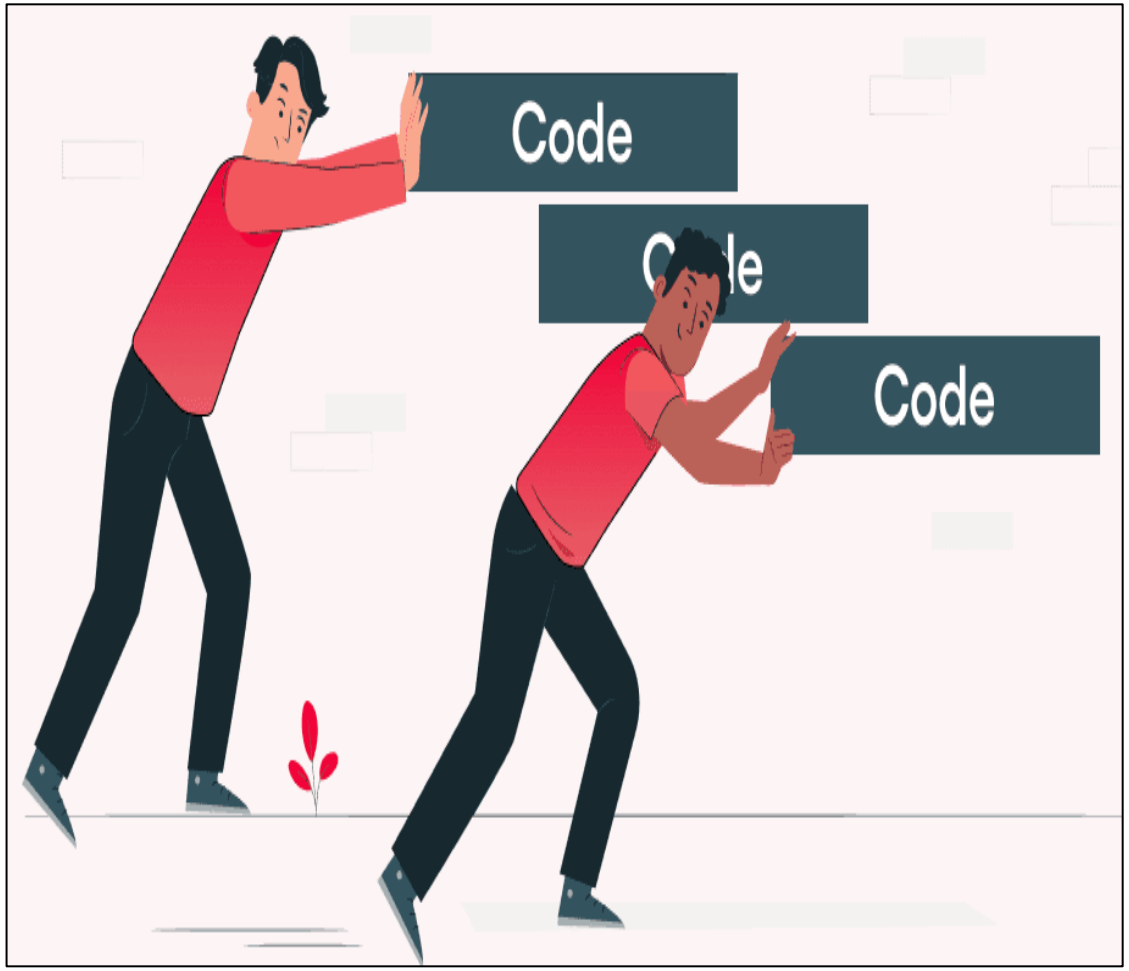
- **Python does not have** a specific syntax for **multi-line comments**, but you can use **triple-quotes ('''** **or """)** to **create a string** that **spans multiple lines**.
- This string is **usually placed at the beginning of a module**, **class**, or **function** and serves as a **docstring** (documentation string) these docstrings can be used to achieve a **similar purpose of documenting code.**

Example : Multi-Line Comment

```
''' This is a multi-line comment using triple-quotes.
It spans multiple lines and is often used as a docstring. '''
```

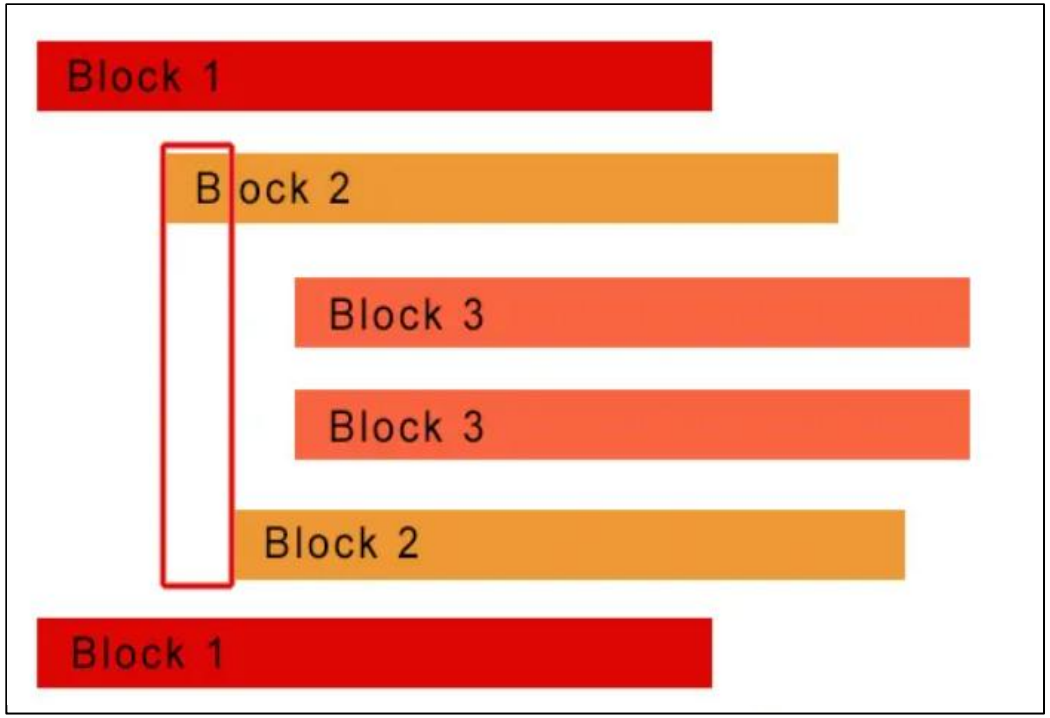
Indentation

- ✓ **Indentation** is **significant** in **Python**, a programming language known for its **readability and simplicity—indentation groups statements** that **belong together**, such as a loop or a conditional statement, etc.,
- ✓ **Python** uses **whitespace** for **indentation** instead of brackets to indicate blocks of code



Indentation

- ✓ The statements which go together must have the **same indentation**. Each such set of **statements is called a block.**
- ✓ In Python, an **indentation error** occurs when **spaces or tabs are not placed properly.**



Error: Indentationerror: unindent does not match any outer Indentation level

Character Set

- ✓ **Character set** means **characters, digits, and symbols** which can be determined by **python language.**
- ✓ The Python character set consists of **Three main elements** as follows**:**
  - **Letters:** These are alphabets ranging from A-Z and a-z (both uppercase and lowercase characters convey different meanings).
  - **Digits:** All the digits from 0 – 9.
  - **Special symbols:** There are a variety of special symbols available in Python like mathematical, logical and relational operators like +,-, *, /, \, ^, %, !, @, #, ^, &, (,), [,], ; and many more.

Character Set

Notes

Python can process all kind of **ASCII** (**A**merican **S**tandard **C**ode for **I**nformation **I**nterchange) & **UNICODE** (Unique / Universal / Uniform character enCoding) characters.

PYTHON DATA TYPES

Introduction

- ✓ The **datatype** specifies the **type of data** that a **variable can store.**
- ✓ In programming, we **encounter many types of values**, some are **sentences,** some are **numbers**, some contain **more than one item,** and so on.
- ✓ In Python, **every value has a data type**, since **everything is an object** in Python, **data types are actually classes, variables being their objects.**
- ✓ **For Example,** The **marks of a student** are a **number,** therefore **Python uses an integer data** type, similarly, the **name of a student is a word**, therefore it is a **string data type**.
- ✓ **The d**ata Types are used to **define the type of a variable and** allocate the **appropriate amount of memory** for **variables**.

Introduction

- ✓ **In Python, don't need to explicitly** specify the **data type** of a **variable** when you **assign a value to it**.
- ✓ Python is a **dynamically typed language**, which means that the **interpreter determines** the **data type** of a **variable** based on the **value it is assigned, and t**his process is called **type inference**.

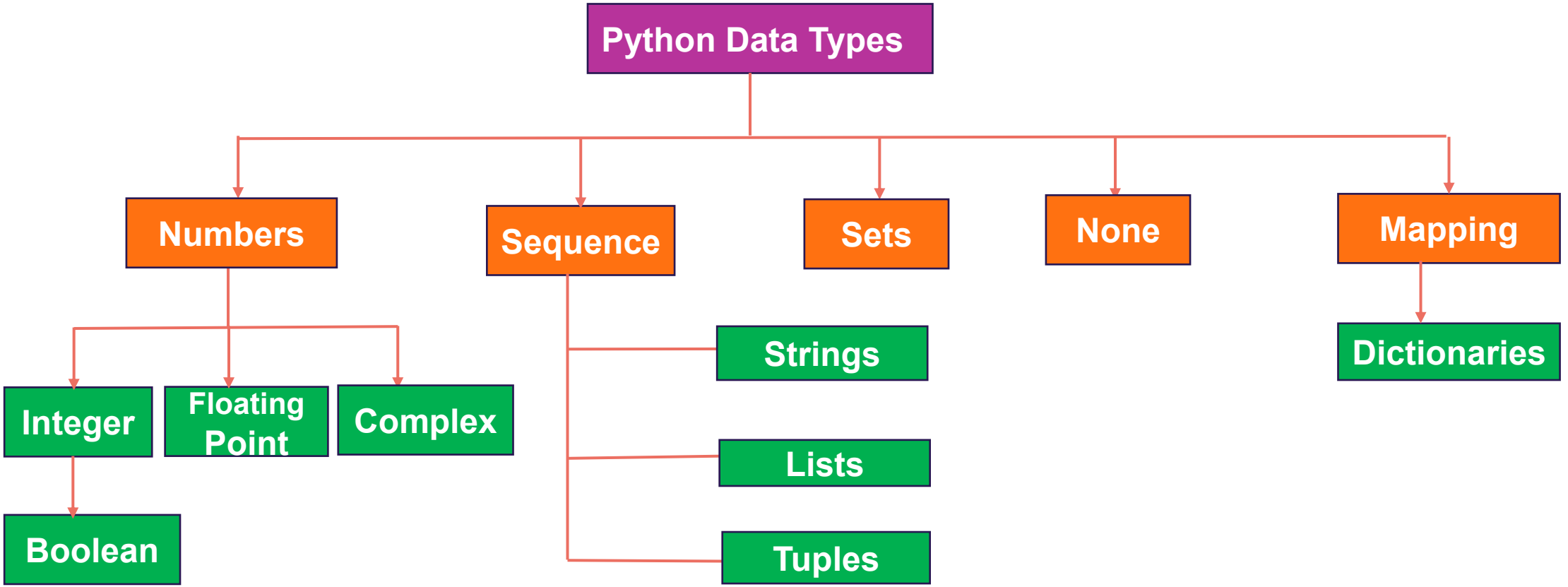
Example:

```
num_1 = 5                 # Python infers that num_1 is of type int
numvalue = 3.14           # Python infers that numvalue is of type float
name = "John"             # Python infers that the name is of type str
```

- ✓ **In the above example, automatically** determines the **data type of x, y**, and **name** based on the values assigned to them.

Types of Data Types

✓ The below image enlists the data types available in python



Number Datatype

- ✓ **Number data type** stores **numerical values** only.
- ✓ It is further classified into **three different types**: **int, float, and complex.**

Type/Class	Descriptions	Examples
int	integer numbers	–12, –3, 0, 125, 2
float	real or floating-point numbers	–2.04, 4.0, 14.23
complex	Complex numbers	3 + 4j, 2 – 2j

- ✓ **Boolean data type** (bool) is a **subtype of integer**. It is a **unique data type**, consisting of **two constants**, **True and False.** Boolean **True value** is **non-zero**, **non-null**, and **non-empty** and **False** is the **value zero.**

Number Data Type

✓ Let us now try with example to **execute a few statements** in **interactive mode** to determine the **data type of the variable** using the **built-in function type()** and this function is used to get the **type of the object.**

```
# Example: Number datatype
num1 = 10
print(type(num1))
num2 = -1210
print(type(num2))
var1 = True
print(type(var1))
# class 'bool'>
```

Number Data Type

```
float1 = -1921.9
print(type(float1))
# class 'float’>
float2 = -9.8 * 10 ** 2
print(float2, type(float2))
# -980.0000000000001 <
# class 'float'>
var2 = -3 + 7.2j
print(var2, type(var2))
# (-3 + 7.2j) <
# class 'complex'>
```

Number Data Type

- ✓ **Variables** of **simple data types** like **integers, float, boolean**, etc., **hold single values**.
- ✓ But **such variables** are **not useful** to **hold a long list of information**, for example, names of the months in a year, names of students in a class, names and numbers in a phone book, or the list of artifacts in a museum.
- ✓ For this, **Python provides** data types like **tuples, lists, dictionaries, and sets** and will be discussed in the following slides.

Sequence Data Type

✓ A **Python sequence** is an **ordered collection of items**, where each item **is indexed by an integer**.

✓ The **three types** of **sequence data types** available in Python are

- **Strings**
- **Lists**
- **Tuples**

Sequence Data Type : string

- ✓ String is a **group of characters** and these characters may be **alphabets, digits, or special characters** including **spaces**.
- ✓ String values are enclosed either in **single quotation marks** (e.g., ‘Hello’) or in **double quotation marks** (e.g., “Hello”).
- ✓ The **quotes** are **not a part** of the **string**, they are used to **mark the beginning and end** of the string for the interpreter.

```
# Example: # Initializing the string with single and double quotes to a variable
str1 = 'Hello Friend’
str2 = "452"
```

Sequence Data Type : list

✓ List is a **sequence of items** separated by **commas** and the **items are enclosed** in **square brackets [].**

Example:

```
#To create a list
list1 = [5, 3.4, "New Delhi", "20C", 45]
#print the elements of the list list1
print(list1)
```

[5, 3.4, 'New Delhi', '20C', 45]

Sequence Data Type : tuples

✓ Tuple is a **sequence of items** separated by **commas** and **items are enclosed** in **parenthesis ( ).**

✓ This is unlike a list, where **values are enclosed in brackets [ ],** and once created, we **cannot change**

Example:

```
#create a tuple tuple1
tuple1 = (10, 20, "Apple", 3.4, 'a')
#print the elements of the tuple tuple1
print(tuple1)
```

(10, 20, "Apple", 3.4, 'a')

Set Data Type

- ✓ **Set** is an **unordered collection** of items **separated by commas** and the items are enclosed in **curly brackets { }.**
- ✓ A **set is similar to a list**, except that it **cannot have duplicate entries,** and once created, elements of a **set cannot be changed.**

Example:

```
#create a set

set1 = {10,20,3.14,"New Delhi"}

print(type(set1))
```

<class 'set'>

Set Data Type

```
print(set1)

{10, 20, 3.14, "New Delhi"}

#duplicate elements are not included in set

set2 = {1,2,1,3}

print(set2)

{1, 2, 3}
```

None Data Type

- ✓ **None** is a **special data type** with a **single value** and used to **signify the absence of value** in a situation.
- ✓ None supports **no special operations**, and it is **neither the same as False nor 0** (zero).

Example:

```
myVar = None
print(type(myVar))
<class 'NoneType'>
print(myVar)
None
```

Mapping : Dictionary

- ✓ **Mapping** is an **unordered data type** in Python.
- ✓ Currently, there is only **one standard mapping data type** in **Python** called a **dictionary** which **permits faster access to data.**
- ✓ **Items** in a **dictionary** are enclosed in **curly brackets { }.**
- ✓ Dictionary in Python holds data items in **key-value pairs** and every **key** is **separated** from its **value using a colon (:) sign**.
- ✓ The **keys** are usually **strings and their values** can be **any data type**.

```
# Example : Creating a dictionary
student = { "name": "Alice", "age": 25, "grade": "A" }
```

Mapping : Dictionary

- ✓ The **key: value** pairs of a dictionary can be **accessed using the key**.
- ✓ In order to **access any value** in the dictionary, we have to **specify its key in square brackets [ ].**

Example :

#create a dictionary

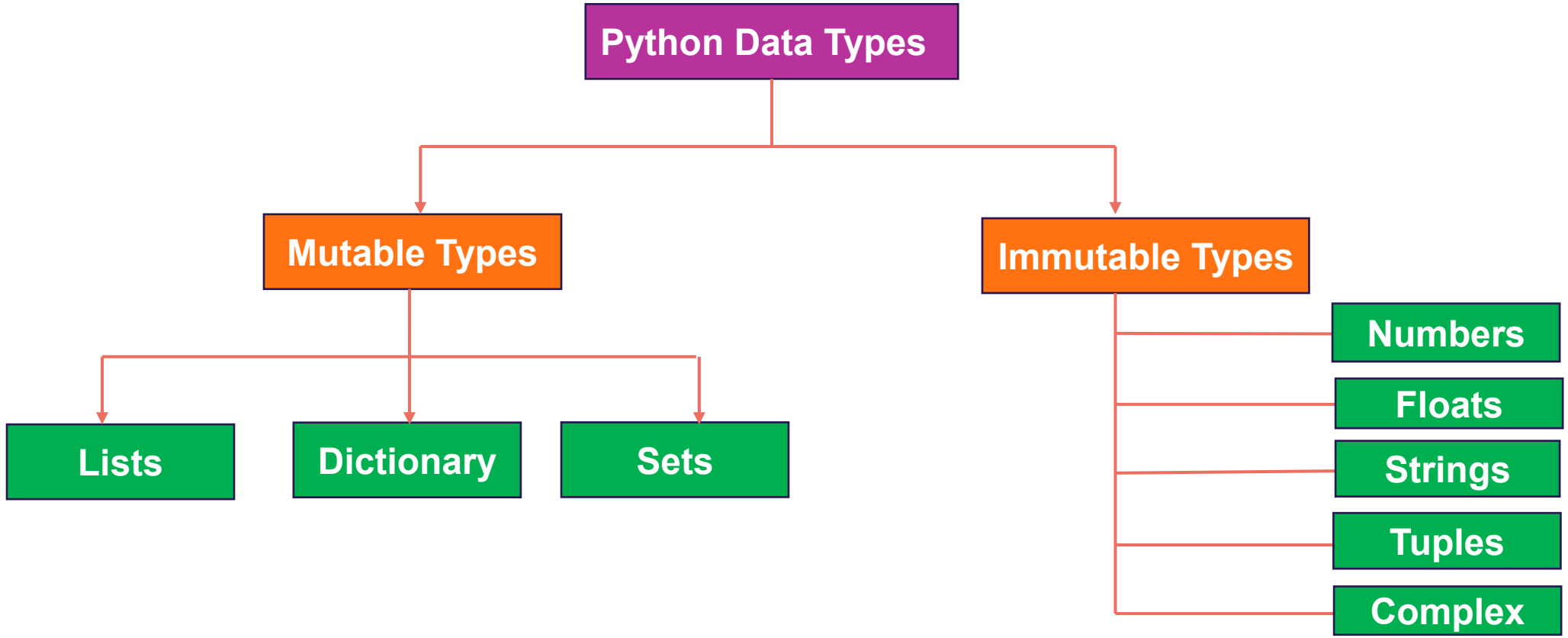
```
>>> dict1 = {'Fruit':'Apple', 'Climate':'Cold', 'Price(kg)' :120}
>>> print(dict1)
{'Fruit': 'Apple', 'Climate': 'Cold', 'Price(kg)': 120}
>>> print(dict1['Price(kg)'])
120
```

Mutable and Immutable Data Types

- ✓ Sometimes we may **require to change** or **update the values** of certain **variables** used in a program.
- ✓ However, for certain data types, Python **does not allow** us to **change the values once a variable** of that type has been **created and assigned values.**
- ✓ **Variables** whose values **can be changed** after they are **created and assigned** are called **mutable**.
- ✓ **Variables** whose **values cannot be changed** after they are created and **assigned** are called **immutable.**
- ✓ When an **attempt is made to update the value** of an **immutable variable**, the **old variable** is **destroyed,** and a **new variable** is created by the **same name in memory.**

Mutable and Immutable Data Types

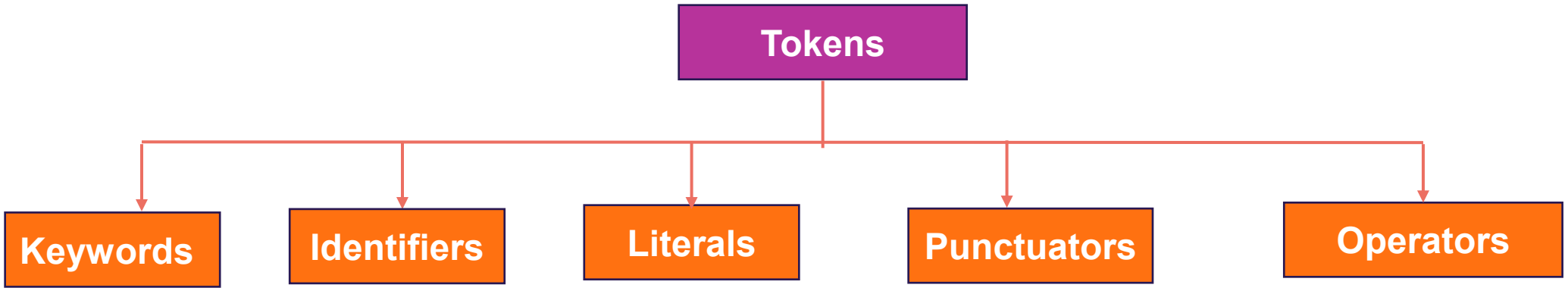
✓ Python data types can be classified into **mutable and immutable** as follows



PYTHON TOKENS

What is Tokens?

✓ **Python** uses **tokens** as its **fundamental building blocks** and it is categorized as below:



Keywords

- ✓ **Keywords** are **reserved words**.
- ✓ Each **keyword** has a **specific meaning** to the **Python interpreter**, and we can use a **keyword** in our **program** only for the **purpose** for which it **has been defined**.
- ✓ As Python is **case-sensitive**, **keywords** must be written exactly as given below:

Python Keywords					
False	if	continue	except	as	with
True	elif	def	finally	in	yield
and	else	return	global	is	
or	for	class	import	lambda	
not	while	del	from	pass	

Identifiers

- ✓ An **identifier** is a **name** given by the **user** for various programming elements like **variables, classes, functions, etc.**
- ✓ It helps to **differentiate one entity from another** and **understand the flow of entities** in code.
- ✓ Python is **case-sensitive** as it **treats upper** and **lower case characters differently.**
- ✓ **Rules of Identifiers:**
  - The **name** should **begin with an uppercase** or a **lowercase alphabet** or an **underscore sign (_)** and this may be followed by any **combination of characters a–z, A–Z, 0–9**, or **underscore (_).** Thus, **an identifier cannot start with a digit.**
  - It can be of **any length**. (However, it is preferred to keep it short and meaningful).
  - We **cannot use special symbols** like !, @, #, $, %,etc., in identifiers.

Identifiers

- It **should not** be a **keyword** or **reserved word.**
- It **must not contain white space**.

Example : Identifiers

Some Valid Identifiers:	**Some Invalid Identifiers:**
name, _abc, file123, STUDENT, _ab_cd, FILE123	123var, class, @username, #, float

Variables

- ✓ A **variable** is a **named storage location** in a **computer's memory** that can **hold a value**.
- ✓ It is often used to **store data that the program**, such as user input, calculations, or intermediate results, **may modify**.
- ✓ **Python** is a **dynamically typed language** that **infers** the **variable type at runtime**. This allows for greater **flexibility** in programming and can make **code easier to read and write**.
- ✓ To **create a variable**, choose a **name for your variable** and use the **assignment operator =** to assign a **value to it**. This process allows you to **store data in memory** and **reference it later** in your program.

Syntax:

```
Variable_name = value
```

Variables

- ✓ **For Example** We can create a variable to represent the cost. You could name this variable "cost_rate" and assign it a value of the total amount allocated for the product.

Example:

```
Cost_rate = 5000          #Assign the value to a variable
```

- ✓ Python allows you to **assign a single value to several variables** simultaneously and also assign **multiple objects to multiple variables.**

Example:

```
num1=num2=20              #Assign a single value to several variables
Cost,count=5000,46        #Assign a multiple object to multiple variables
```

Variables

- ✓ There are a **few rules** we need to follow while **creating a variable** in Python:
  - Please ensure that the variable name only includes **letters, numbers, and underscores. No** other **special characters** are permitted like @,#,&.
  - The **variable name** should either begin with an **Uppercase(A to Z)** or **Lowercase(a to z)** character or an **underscore(_).**
  - The **variable name** must **not begin** with **a number**.
  - The **variable name** should be **descriptive** and **easy to understand**.
  - The **variable name** is **case-sensitive**, meaning "age" and "Age" are two different variables.
  - **Avoid** using **Python-reserved words** as variable names.

Variables

✓ Below are some valid and invalid variable name:

Example: Valid variable

harry

harry_potter

harryPotter

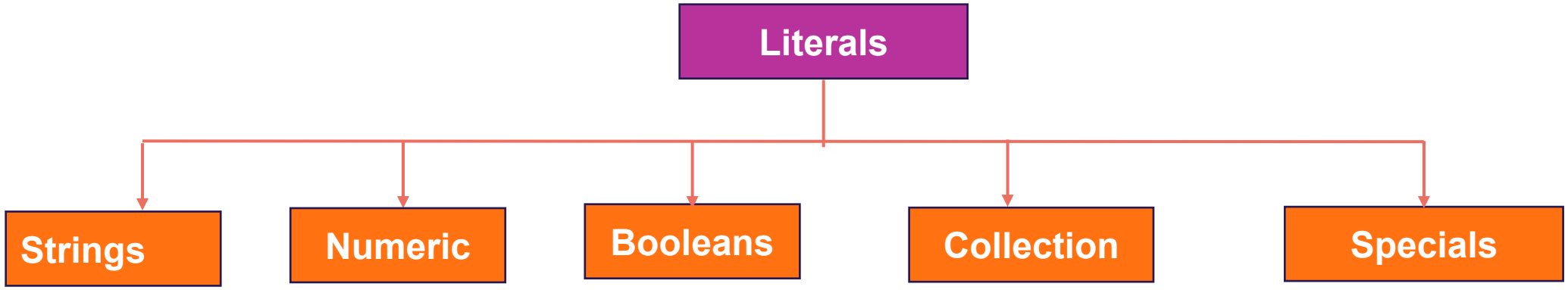
harryPotter2

Example: Invalid variable

```
2harry          # cannot start with a number
harry-potter    # cannot contain a hyphen
h@rry           # cannot contain a special character
harry potter    #Spaces are not allowed in var
names
```

Literals

✓ **Literals** (often referred to as constants) are **data items** that **never change** their **value** during a **program run**.



Literals : String

- ✓ **String** is the **collection of the characters** surrounded by **single quotes, double quotes, triple quotes**, or their combination.
- ✓ The **computer** does **not understand the characters**; internally, it **stores manipulated character as the combination of the 0's and 1's** and each **character** is **encoded in the ASCII** or **Unicode character**.

Example:

```
'What's your name?'                          # Single Quotes
"What's your name?"                          # Double Quotes
'''This is a multi-line string. This is the first line.
        This is the second line.
        "What's your name?," I asked.
        He said "Bond, James Bond."
        '''                                  # Triple Quotes
```

Literals : Numeric

- ✓ These are the Literals that are used to **define values in digits or Numbers.**
- ✓ Some of the Numeric Literals in Python are **Int, Float, Complex, Octal, Hexadecimal**.

Example:

```
c = 23                  # Integer Literal          print("Value of c is ",c)
e = 23.657787           # Floating Point Literal   print("Value of e is ",e)
f = 23+5j               # Complex Literal          print("Value of f is ",f.real,f.imag)
g = 0x18d               # Hexadecimal Literal       print("Value of g is ",g)
d = 0o257               # Octal Literal            print("Value of d is ",d)
# Octal and Hexadecimal notations will automatically print their corresponding decimal values.
```

Literals : Boolean

✓ A **Boolean literal** can have any of the **two values**: **True (1) or False (0).**

Example:

```
x = (1 == True)          print("x is", x)
y = (1 == False)         print("y is", y)
a = True + 4             print("a:", a)
b = False + 10           print("b:", b)
```

Literals : Collection

- ✓ The **collection** is made up of **different types of primitive data type** objects that are **grouped** to form a **single entity.**
- ✓ In Python, **Collection includes List, Set, Tuple, and Dictionary**, etc. (We will discuss briefly in Data structure)

Literals : Special

- ✓ Python has a **special kind** of literal known as **None** which simply means that a **field has not been created.**
- ✓ **None** is a **pre-defined keyword** but because it **works as a literal** also, therefore considered as a Special Literal.

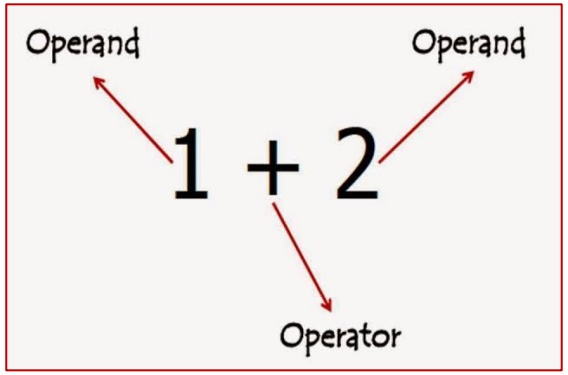
Example:

```
None                    # a None literal representing null
```

Operators

✓ An **operator** is used to **perform specific mathematical** or **logical operations on values**.

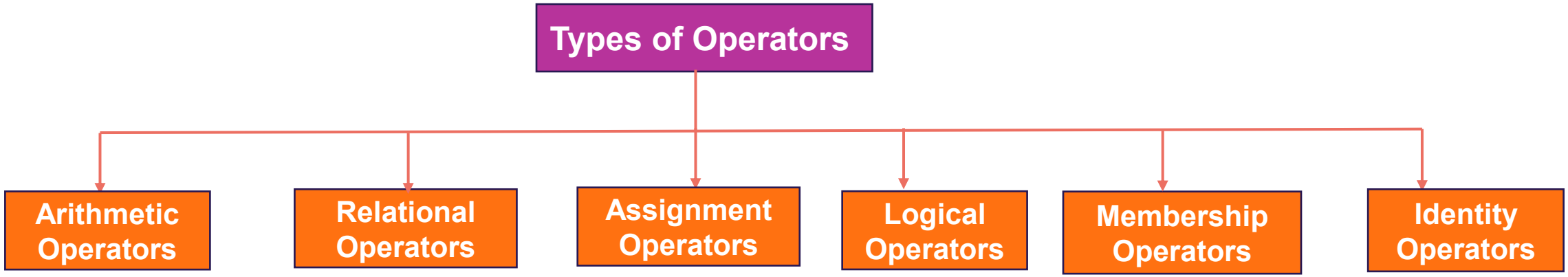
✓ The **values** that the **operators work** on are called **operands.**



✓ For **example,** in the **expression 1 + 2**, the **value 1, and 2 are operands** and the **+** (plus) sign is an **operator.**

Types of Operators

✓ Python supports **several kinds of operators** whose categorization is briefly explained as follows



Types of Operators : Arithmetic Operators

✓ **Arithmetic operators** that are used to perform the **four basic arithmetic operations** as well as modular division, floor division, and exponentiation.

Operator	Operation	Description	Example

+	Addition	✓Adds the two numeric values on either side of the operator
		✓This operator can also be used to concatenate two strings on either side of the operator
			>>> num1 = 5
			>>> num2 = 6
			>>> num1 + num2
			11
			
			>>> str1 = "Hello"
			>>> str2 = "India"
			>>> str1 + str2
			'HelloIndia'

Types of Operators : Arithmetic Operators

Operator	Operation	Description	Example

-	Subtraction	✓ Subtracts the operand on the right from the operand on the left	>>> num1 = 5
			>>> num2 = 6
			>>> num1 - num2
			-1

*	Multiplication	✓Multiplies the two values on both side of the operator.
		✓Repeats the item on left of the operator if first operand is a string and second operand is an integer value.
			>>> num1 = 5
			>>> num2 = 6
			>>> num1 * num2
			30
			
			>>> str1 = 'India'
			>>> str1 * 2
			'IndiaIndia'

Types of Operators : Arithmetic Operators

Operator	Operation	Description	Example

/	Division	✓Divides the operand on the left by the operand on the right and returns the quotient	>>> num1 = 8
			>>> num2 = 4
			>>> num2 / num1
			0.5

%	Modulus	✓Divides the operand on the left by the operand on the right and returns the remainder.	>>> num1 = 13
			>>> num2 = 5
			>>> num1 % num2
			3

Types of Operators : Arithmetic Operators

Operator	Operation	Description	Example

//	Floor Division	✓Divides the operand on the left by the operand on the right and returns the quotient by removing the decimal part.
		✓It is sometimes also called integer division.
			>>> num1 = 13
			>>> num2 = 4
			>>> num1 // num2
			3
			>>> num2 // num1
			0

**	Exponent	✓Performs exponential (power) calculation on operands.That is , raise the operand on the left to the power of the operand on the right	>>> num1 = 3
			>>> num2 = 4
			>>> num1 ** num2
			81

Types of Operators : Relational Operators

- ✓ **Relational operator compares** the **values of the operands** on **either side** and determines the relationship among them.
- ✓ Assume the Python variables num1 = 10, num2 = 0, num3 = 10, str1 = "Good", str2 = "Afternoon" is used as an example to illustrate the relational operators as follows:

Operator	Operation	Description	Example
==	Equals to	✓If the values of two operands are equal, then the condition is True, otherwise it is False	>>> num1 == num2 False >> str1 == str2 False

Types of Operators : Relational Operators

Operator	Operation	Description	Example

!=	Not equal to	✓If values of two operands are not equal, then condition is True, otherwise it is False	>>> num1 != num2
			True
			>>> str1 != str2
			True
			>>> num1 != num3
			False

>	Greater than	✓If the value of the left-side operand is greater than the value of the right-side operand, then condition is True, otherwise it is False	>>> num1 > num2
			True
			>>> str1 > str2
			True

<	Less than	✓If the value of the left-side operand is less than the value of the right-side operand, then condition is True, otherwise it is False	>>> num1 < num3
			False
			>>> str2 < str1
			True

Types of Operators : Relational Operators

Operator	Operation	Description	Example

>=	Greater than or equal to	✓If the value of the left-side operand is greater than or equal to the value of the right-side operand, then condition is True, otherwise it is False	>>> num1 >= num2
			True
			>>> num2 >= num3
			False
			>>> str1 >= str2
			True

<=	Less than or equal to	✓If the value of the left operand is less than or equal to the value of the right operand, then is True otherwise it is False	>>> num1 <= num2
			False
			>>> num2 <= num3
			True
			>>> str1 <= str2
			False

Types of Operators : Assignment Operators

✓ **Assignment operator** assigns or changes the **value of the variable on its left**.

Operator	Description	Example

=	✓Assigns value from right-side operand to left-side operand	>>> num1 = 2
		>>> num2 = num1
		>>> num2
		2
		>>> country = 'India'
		>>> country
		'India'

Types of Operators : Assignment Operators

Operator	Description	Example

+=	✓It adds the value of right-side operand to the left-side operand and assigns the result to the left-side operand
	Note: x += y is same as x = x + y
		>>> num1 = 10
		>>> num2 = 2
		>>> num1 += num2
		>>> num1
		12
		>>> num2
		2
		>>> str1 = 'Hello'
		>>> str2 = 'India'
		>>> str1 += str2
		>>> str1
		'HelloIndia’

Types of Operators : Assignment Operators

Operator	Description	Example

- =	✓It subtracts the value of right-side operand from the left-side operand and assigns the result to left-side operand
	Note: x -= y is same as x = x - y
		>>> num1 = 10
		>>> num2 = 2
		>>> num1 -= num2
		>>> num1
		8

*** =**	✓It multiplies the value of right-side operand with the value of left-side operand and assigns the result to left-side operand
	Note: x *= y is same as x = x * y
		>>> num1 = 2
		>>> num2 = 3
		>>> num1 *= 3
		>>> num1
		6
		>>> a = 'India'
		>>> a *= 3
		>>> a
		'IndiaIndiaIndia'

Types of Operators : Assignment Operators

Operator	Description	Example

- =	✓It subtracts the value of right-side operand from the left-side operand and assigns the result to left-side operand
	Note: x -= y is same as x = x - y
		>>> num1 = 10
		>>> num2 = 2
		>>> num1 -= num2
		>>> num1
		8

*** =**	✓It multiplies the value of right-side operand with the value of left-side operand and assigns the result to left-side operand
	Note: x *= y is same as x = x * y
		>>> num1 = 2
		>>> num2 = 3
		>>> num1 *= 3
		>>> num1
		6
		>>> a = 'India'
		>>> a *= 3
		>>> a
		'IndiaIndiaIndia'

Types of Operators : Assignment Operators

Operator	Description	Example

/=	✓It divides the value of the left-side operand by the value of the right-side operand and assigns the result to the left-side operand
	Note: x /= y is same as x = x / y
		>>> num1 = 6
		>>> num2 = 3
		>>> num1 /= num2
		>>> num1
		2.0

%=	✓It performs modulus operation using two operands and assigns the result to left-side operand
	Note: x %= y is same as x = x % y
		>>> num1 = 7
		>>> num2 = 3
		>>> num1 %= num2
		>>> num1
		1

Types of Operators : Assignment Operators

Operator	Description	Example

//=	✓It performs floor division using two operands and assigns the result to left-side operand
	Note: x //= y is same as x = x // y
		>>> num1 = 7
		>>> num2 = 3
		>>> num1 //= num2
		>>> num1
		2

**=	✓It performs exponential (power) calculation on operators and assigns value to the left-side operand
	Note: x **= y is same as x = x ** y
		>>> num1 = 2
		>>> num2 = 3
		>>> num1 **= num2
		>>> num1
		8

Types of Operators : Logical Operator

- ✓ The logical operator evaluates to **either True or False** based on the logical operands on either side.
- ✓ There are **three logical operators** supported by Python and these operators are **and, or, not** operators that are to be written in **lowercase only.**
- ✓ By default, **all values are True except None**, **False, 0 (zero), empty collections "", (), [], {},** and a few other special values. So if we **say num1 = 10, num2 = -20**, then both **num1 and num2 are logically True.**

Types of Operators : Logical Operator

Operator	Operation	Description	Example

and	Logical AND	✓If both the operands are True, then condition becomes True	>>> True and True
			True
			>>> num1 = 10
			>>> num2 = -20
			>>> bool(num1 and num2)
			True
			>>> True and False
			False
			>>> num3 = 0
			>>> bool(num1 and num3)
			False
			>>> False and False
			False

Types of Operators : Logical Operator

Operator	Operation	Description	Example

or	Logical OR	✓ If any of the two operands are True, then the condition becomes True	>>> True or True
			True
			>>> True or False
			True
			>>> bool(num1 or num3)
			True
			>>> False or False
			False

not	Logical NOT	✓Used to reverse the logical state of its operand	>>> num1 = 10
			>>> bool(num1)
			True
			>>> not num1
			>>> bool(num1)
			False

Types of Operators : Identity Operator

✓ **Identity operators** are used to determine whether the **value of a variable** is of a **certain type or not**.

✓ Identity operators can also be used to determine whether **two variables** are **referring to the same object or not.** There are **two identity operators** as follows:

Operator	Description	Example

is	✓Evaluates True if the variables on either side of the operator point towards the same memory location and False otherwise. var1 is var2 results to True if id(var1) is equal to id(var2)	>>> num1 = 5
		>>> type(num1) is int
		True
		>>> num2 = num1
		>>> id(num1)
		1433920576
		>>> id(num2)
		1433920576

Types of Operators : Identity Operator

Operator	Description	Example
is not	✓Evaluates to False if the variables on either side of the operator point to the same memory location and True otherwise. Var1 is not var2 results to True if id(var1) is not equal to id(var2)	>>> num1 is not num2 False

Types of Operators : Membership Operator

✓ **Membership operators** are used to **check if a value** is a **member of the given sequence or not**.

Operator	Description	Example

in	✓ Returns True if the variable/value is found in the specified sequence and False otherwise	>>> a = [1,2,3]
		>>> 2 in a
		True
		>>> '1' in a
		False

not in	✓ Returns True if the variable/value is not found in the specified sequence and False otherwise	>>> a = [1,2,3]
		>>> 10 not in a
		True
		>>> 1 not in a
		False

Expression

What is Expression?

- ✓ An **Expression** is defined as a **combination of constants, variables, and operators.**
- ✓ An expression always **evaluates to a value.**
- ✓ A **value** or a **standalone variable** is also considered **as an expression** but a **standalone operator is not an expression.**

Example : # Valid Expressions

(i) 100

(ii) num

(iii) num – 20.4

(iv) 3.0 + 3.14

(v) 23/3 - 5 * 7 (14 -2)

(vi) "Global" + "Citizen"

What is Expression?

Operator Precedence

- It **determines which operator** is performed **first** in an **expression** with more than one operator with different precedence.

Operators Associativity

- It is used when **two operators** of the **same precedence** appear in an expression. Associativity can be either **Left to Right or Right to Left**.

Operator Precedence and Associativity

OPERATOR	DESCRIPTION	ASSOCIATIVITY
( )	Parentheses	left-to-right
**	Exponent	right-to-left
* / // %	Multiplication/division/floor division /modulus	left-to-right
+ –	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right

Operator Precedence and Associativity

OPERATOR	DESCRIPTION	ASSOCIATIVITY
is, is not in, not in	Identity Membership operators	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
|	Bitwise inclusive OR	left-to-right
not	Logical NOT	right-to-left
and	Logical AND	left-to-right
or	Logical OR	left-to-right

Operator Precedence and Associativity

OPERATOR	DESCRIPTION	ASSOCIATIVITY
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left

Example #1

How will Python evaluate the following expression?

Expression :20 + 30 * 40

Solution:

```
#Step 1
= 20 + (30 * 40)
#precedence of * is more than that of +
#Step 2
= 20 + 1200
#Step 3
= 1220
```

Example #2

How will Python evaluate the following expression?

Expression:20 - 30 + 40

Solution: The two operators (–) and (+) have equal precedence. Thus, the first operator, i.e., subtraction is applied before the second operator, i.e., addition (left to right).

#Step 1

= (20 – 30) + 40

#Step 2

= -10 + 40

#Step 3

= 30

Example #3

How will Python evaluate the following expression?

Expression:(20 + 30) * 40

Solution:

#Step 1

= (20 + 30) * 40

#using parenthesis(), we have forced precedence of + to be more than that of *

#Step 2

= 50 * 40

#Step 3

=2000

Example #4

How will Python evaluate the following expression?

Expression:15.0 / 4 + (8 + 3.0)

Solution:

#Step 1

= 15.0 / 4 + (8.0 + 3.0)

#Step 2

=15.0 / 4.0 + 11.0

#Step 3

=3.75 + 11.0

#Step 4

=14.75

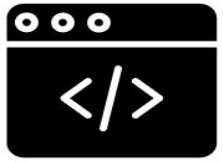
Type Conversion

What is Type Conversion?

- ✓ The **process of converting** the **value of one data type (integer, string, float, etc.) to another data type** is called **type conversion.**
- ✓ Python has **two types** of type conversion as follows:
  - **Implicit Type Conversion**
  - **Explicit Type Conversion**

Implicit Type Conversion

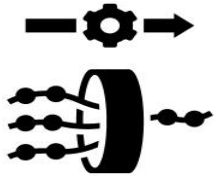
✓ In Implicit type conversion, Python **automatically converts** one data type to another data type.



Program

```
#Implicit type conversion from int to float wherenum1 is an integer and num2 is a float
num1 = 10
num2 = 20.0
#sum1 is sum of a float
sum1 = num1 + num2
print(sum1)
print(type(sum1))
```

Implicit Type Conversion



Sample Input and Output

```
30.0
<class 'float'>
```

- ✓ **One may wonder why was the float value not converted to an integer instead.**
- ✓ This is due to **type promotion** that allows **performing operations** (whenever possible) by **converting data into a wider-sized data type without any loss of information.**
- ✓ In the case of **adding an integer and a float**, Python **promotes the integer to a float** and then **performs the addition**.

Implicit Type Conversion

- ✓ This is done to **avoid potential loss of precision**, as **converting a float to an integer** can result in **truncating the fractional part** of the **float**.
- ✓ This **behavior** is part of **Python's dynamic typing** and **type coercion system**, which aims to provide a **balance between convenience** and **preserving data integrity**.
- ✓It's essential for **programmers** to be **aware** of this **behavior** to **avoid unexpected results** and **handle type conversions explicitly when necessary**.

Explicit Type Conversion

- ✓ **Explicit conversion**, also called **type casting** happens when **data type conversion** takes place because the **programmer forced** it in the program.
- ✓ The general form of an explicit data type conversion is

```
(new_data_type) (expression)
```

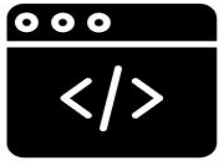
- ✓ With explicit type conversion, there is a **risk of loss of information** since we are **forcing an expression to be of a specific type.**
- ✓ For example, converting a **floating value of x = 20.67** into an **integer type**, that is int(x) will **discard the fractional part .67.**

Explicit Type Conversion

✓ Following are some of the functions in Python that are used for explicitly converting an expression or a variable to a different type.

Function	Description
int(x)	Converts x to an integer
float(x)	Converts x to a floating-point number
str(x)	Converts x to a string representation
chr(x)	Converts ASCII value of x to character
chr(x)	returns the character associated with the ASCII code x

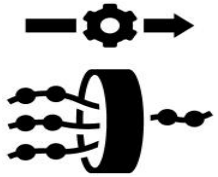
Explicit Type Conversion



Program

```
#Explicit type conversion from int to float
num1 = 10
num2 = 20
num3 = num1 + num2
print(num3)
print(type(num3))
num4 = float(num1 + num2)
print(num4)
print(type(num4))
```

Explicit Type Conversion



Sample Input and Output

```
30
<class 'int'>
30.0
<class 'float'>
```

Input and Output

input() function

- ✓ Sometimes, a **program needs to interact** with the **user** to get some **input data** or **information** from the end user and **process** it to give the **desired output**.
- ✓ In Python, we have the **input()** function for taking the **user input** that prompts the **user to enter data** and accepts the **input as a string**.
- ✓ The user may **enter a number** or a **string** but the **input() function treats them as strings only**.

input() function

✓ The **syntax** for input() is:

```
input ([Prompt])
```

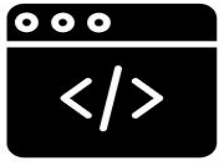
✓ **Prompt** is the **string** we may like to **display on the screen** prior to taking the input, and it is optional.

✓ When a **prompt is specified**, **first it is displayed on the screen after** which the **user can enter data**.

✓ The **input()** takes exactly what is **typed from the keyboard**, **converts it into a string**, and **assigns it to the variable** on the left-hand side of the **assignment operator (=).**

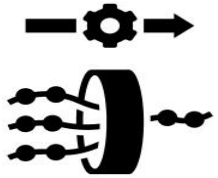
✓ Entering data for the input function is terminated by **pressing the enter key**.

input() function



Program

```
# Using Input()
fname = input("Enter your first name: ")
age = input("Enter your age: ")
print(type(age))
```



Sample Input and Output

```
Enter your first name: smartcliff
Enter your age: 2
<class 'str'>
```

✓ In this program, the variable ‘fname’ will get the string ‘Smartcliff’, similarly, the variable ‘age’ will get ‘2’ as a string.

input() function

- ✓ We can **typecast or change the datatype** of the string data accepted from the **user** to an **appropriate numeric value.**

```
# Using Input()
fname = input("Enter your first name: ")
age = int(input("Enter your age: "))
print(type(age))
```

print() function

✓ The **print() function** in Python is used to **display output to the console or terminal.** It can take **one or more arguments and print** them to the **standard output.**

```
# priniting a string
print("Hello, World!")
# Printing multiple values
x = 5
y = 10
print("The value of x is", x, "and the value of y is", y)
# Formatting with f-strings (Python 3.6 and later)
name = "Alice"
age = 30
print(f"My name is {name} and I am {age} years old.")
```

print() function

```
# Using the sep and end parameters
print("apple", "orange", "banana", sep=", ", end=".\n")
```

✓ The above snippet will print: **apple, orange, banana.**

- The **sep parameter** is used to **specify the separator between the values** (default is a space).
- The **end parameter** is used to **specify what character or string is printed** at the end (default is a newline \n).


THANK YOU
SmartCliff
Career Mobility Solutions